
TomasekValla Filestream System

A Whitepaper on Sovereign File Distribution

`files.tomasekvalla.cz`
Version 1.0 — June 2026

This document describes the architecture, philosophy, and technical implementation of the TomasekValla Filestream System (TVFS) — a self-hosted, open-source file distribution platform built on the principle that your files should be controlled by you, served by you, and deleted by you.

No cloud. No compression. No surveillance. No corporation.

1.

Introduction

There is something deeply wrong with the way we share files in 2026. We have collectively handed over our photographs, our audio recordings, our videos, our documents — the digital artifacts of our lives and work — to a small number of corporations whose business model depends on owning, analyzing, and monetizing everything that passes through their servers.

We were promised convenience. We got surveillance.

We were promised storage. We got compression, transcoding, and quality degradation — all silently applied, all justified by "optimizing the experience" for the platform's bandwidth budget, never for the integrity of your file.

TVFS is a rejection of that bargain. It is a file distribution system that runs on hardware you control, serves files you own, at speeds that don't apologize for file size, and deletes your data on your schedule — not the platform's retention policy.

2.

The Adversaries

To understand why TVFS exists, it helps to name the problems it is designed to replace. These are not abstract concerns — they are the daily friction of anyone who takes their files seriously.

2.1 Corporate Cloud Storage

Google Drive, Dropbox, OneDrive, iCloud. These platforms share a common architecture: your files live on their infrastructure, subject to their terms of service, readable by their systems, and potentially accessible to advertisers, data brokers, or government requests. You are not the customer. You are the inventory.

Beyond the privacy problem, there is a quality problem. Upload a high-resolution video to Google Drive and share it — the recipient downloads your file intact. Fine. But upload it to Google Photos and the story changes. Recompress to Google's preferred codec, strip metadata, resize for "storage savings." The file that comes out the other end is not the file that went in.

2.2 Social Media Platforms

Discord, Instagram, Twitter/X, WhatsApp, Facebook — every one of these platforms aggressively reprocesses media. Discord recompresses images and caps video quality. Instagram crushes photos to roughly 1080px and applies its own sharpening. WhatsApp strips EXIF and reduces video bitrate by factors of ten or more.

The damage is not always visible at a glance. A casual scroll looks fine. But if you are a photographer, an audio engineer, a videographer, or anyone who cares about the technical integrity of a file, these platforms are the enemy of your work.

Beyond compression, these platforms retain your content. Deleted posts surface in caches. Files uploaded years ago remain on servers you have no access to. Your data outlives your intention.

2.3 The Compression Agenda

Compression is not neutral engineering. It is a business decision made at your expense. Bandwidth costs money. Storage costs money. If a platform can cut a file's size by 60% before storing or serving it, that's 60% of their infrastructure cost that you are paying with quality loss.

TVFS does not touch your files. What goes in comes out. The bit pattern you upload is the bit pattern that gets served. This is not a feature — it is the minimum baseline of respect for the files you created.

3.

No Compression, High Fidelity

TVFS stores every file exactly as uploaded. There is no transcoding pipeline, no format normalization, no silent optimization pass. A 4K RAW video file arrives as a 4K RAW video file. A 192kHz/24-bit FLAC arrives intact. A 50MB PNG with embedded color profiles and metadata is served as those exact 50MB.

Files are streamed directly from the server at whatever speed the connection supports. The TVFS server is capable of delivering content at over 300 Mbps — fast enough that a 1GB file is in the recipient's browser within seconds. Live streaming of high-bitrate video directly from the cloud storage is not just possible, it is the default behavior.

This is what file sharing looked like before platforms decided they knew better than you what your files should be.

4.

Open Source as a Trust Mechanism

TVFS is fully open source. The frontend — every HTML template, every CSS rule, every client-side script — is visible to anyone who loads the page. View source is not obfuscated, minified beyond reading, or hidden behind a build artifact. What you see is what runs.

The backend is available on GitHub. Every route, every authentication handler, every rate limiter, every file operation can be audited by anyone with a browser and a few minutes.

There is no proprietary logic. There is nothing hidden.

This is not idealism. It is the only epistemically sound position when it comes to software that handles your files. Closed-source file handling is a trust assertion with no evidence. Open-source file handling is a verifiable claim. The difference matters.

Open source is also the only long-term guarantee of software quality. Corporate SaaS products get deprecated, acquired, pivoted, or simply shut down. Open source code runs as long as someone wants to run it. TVFS can be forked, self-hosted, adapted, and extended without permission from anyone.

5.

Ownership of Data

When you upload a file to TVFS, it lives on a server you control. Not on infrastructure owned by a cloud provider under a shared tenancy agreement. Not in a data center whose jurisdiction you cannot verify. On hardware you chose, in a location you decided.

Files are assigned an expiration period at upload time — currently between 1 and 14 days for batch uploads. When that period expires, the file is deleted. Not archived. Not moved to cold storage for "compliance purposes." Deleted. The registry entry is removed. The bytes are gone.

This is not a limitation. It is a deliberate design choice. Files on the internet that live forever create risk — for the sender, for the recipient, for anyone who might be depicted in them. Ephemeral by default is a security posture, not a technical shortcut.

5.1 Cloudflare Protection

The TVFS server sits behind Cloudflare. Among the protections this provides, one is particularly relevant in 2026: AI crawler blocking. Large language model training pipelines aggressively crawl the public internet, consuming any content they can reach to expand training datasets. TVFS files are blocked from these crawlers at the network edge.

Your files are not training data. They are not raw material for someone else's commercial product. Cloudflare's bot management enforces this boundary so that the content you share does not get ingested into a model you never consented to contribute to.

Combined with automatic expiration, this means TVFS files have a clearly bounded existence: they are available to their intended recipients for a defined period, invisible to crawlers during that period, and then gone.

6.

Technical Architecture

6.1 Password Hashing and Authentication

Access to TVFS is gated by a tiered password system. Passwords are never stored in plaintext. Each password is hashed with SHA-256 at the moment of entry, and only the hash is stored in the server configuration. Comparison during authentication is hash-to-hash — the raw password never persists beyond the single request that produces it.

Two access tiers exist. Tier 1 grants standard upload and download capabilities. Tier 2 grants elevated permissions. Upon successful authentication, the server issues an `HttpOnly`, `Secure`, `SameSite=None` cookie containing the hashed token. This cookie authenticates subsequent requests without re-prompting for a password, with a configurable lifetime of 1 to 30 days.

Environment variables handle all sensitive configuration. API keys, hashed passwords, and server parameters live in `ENV` — not in code, not in committed configuration files. This is standard practice and TVFS treats it as non-negotiable.

6.2 Rate Limiting

Every sensitive endpoint is protected by in-memory, per-IP rate limiting. The authentication endpoint allows a maximum of 10 requests per 15-minute window. Upload initiation is limited to 20 requests per minute. Chunk endpoints impose their own per-IP limits.

Rate limit state is maintained in a Node.js Map with automatic expiry: entries older than 60 seconds past their window reset time are pruned every 30 seconds. This prevents unbounded memory growth without introducing an external dependency like Redis.

When a limit is exceeded, the server responds with HTTP 429 and a `Retry-After` header indicating exactly how many seconds the client should wait. No ambiguity, no exponential backoff guessing required.

6.3 Chunked Upload

Large files — up to 5 GB — are uploaded in chunks of up to 32 MB each. The client initiates a chunked upload session, receives a unique upload ID, and then sends individual chunk requests in parallel or series. Each chunk is written to a temporary directory on the server under a content-addressed filename.

A per-upload-ID manifest tracks which chunks have arrived, their byte offsets, and the total expected size. Access to the manifest is serialized via a promise-based mutex to prevent race conditions when multiple chunks arrive simultaneously. When all chunks are confirmed present, the server assembles them in order into the final file and cleans up the temporary directory.

Unfinished chunk uploads are automatically swept after 24 hours by a cleanup interval, preventing orphaned temporary data from accumulating on disk.

6.4 Resumable Upload

If a chunked upload is interrupted — by a network drop, a browser crash, or any other failure — it does not need to restart from zero. The client can query the server for the status of any upload ID. The server consults the manifest and reports which chunks have already been received.

The client then resumes by sending only the missing chunks. For a large file on a flaky connection, this is the difference between a viable upload and an exercise in frustration.

6.5 Batch Creator

Multiple files can be grouped into a named batch at upload time. The server generates a batch ID, associates uploaded files with it via a JSON registry, and upon batch completion, generates a static HTML landing page for the batch.

The landing page lists every file with its size, provides per-file copy buttons for both raw URLs and formatted markdown links, and includes the batch expiration date. It is hosted at a stable URL and requires no authentication to access — making it suitable for sharing with recipients who have no TVFS account.

6.6 ZIP Downloader

From a batch landing page, recipients can request a ZIP archive of all files in the batch. The server streams the batch files through an archiver pipeline and writes the resulting ZIP to the download directory. The ZIP is then served as a normal file with its own expiration aligned to the source batch.

Individual file downloads are also available directly, without requiring ZIP assembly. The ZIP option exists as a convenience for batches where downloading files individually would be tedious.

6.7 File Registry

Every uploaded file and every batch is tracked in a persistent JSON registry written to disk. The registry records file IDs, original names, stored names, paths, MIME types, sizes, upload timestamps, and expiration times. Writes are debounced with a 2-second delay to avoid thrashing the filesystem on rapid sequential uploads, with a synchronous fallback for cases where an immediate write is required (such as abort handlers).

7.

Conclusion

The centralization of file sharing was not inevitable. It was a set of economic incentives that happened to align around a handful of large platforms at a particular moment in the internet's history. Those platforms extracted enormous value — in data, in attention, in lock-in — from

the files people entrusted to them.

TVFS is a stake in the ground against that arrangement. It is not a product. It is not a service with a pricing page and a sales funnel. It is software that runs on hardware you own and does exactly what you tell it to do with your files. Nothing more, nothing less.

The files you share are yours. The server that serves them is yours. The schedule on which they are deleted is yours. No corporation co-owns your data as a condition of transit. No algorithm recompresses your work to save money on someone else's bandwidth bill. No AI crawler trains on your content because you happened to use a public hosting platform.

That should be the default. TVFS makes it one.

files.tomasekvalla.cz
TomasekValla Filestream System
